

Executive Summary

- Tech stack stays as-is: Flutter, Node.js+TypeScript, PostgreSQL+PostGIS, Google Maps, Socket.IO, Firebase Auth+JWT. Two schema decisions get pulled forward to now (Section 6) so fire brigades and org ratings don't force a rebuild later.
- MVP stays ambulance-only. Fire brigade moves out of the blueprint's old "future/production" bucket (Part C) and into the "full FYP" scope, landing at external evaluation, not squeezed into the 4-week MVP window.
- Both ambulance-request paths are now in scope for MVP: automatic (in-app) and manual (caller phones the org, org submits on the caller's behalf). Both write into the same `ServiceRequest` pipeline and status state machine.
- Closest-vs-rating question is resolved: closest-available is the default for every request. A rating-based alternative only appears when the user explicitly marks a request non-urgent, and even then it's capped by a distance/time ceiling.
- Cross-org MOU failover is now a Must-have: if an assigned org goes busy before confirming, the system silently reassigns to the next-closest available org.
- Data model needs two changes made now, before MVP code is written, so fire brigades and org ratings don't force a rebuild later: a generic `resourceType` on the ambulance/fire table, and a rating field on `Organization`.
- Timeline: MVP by end Sept/1st week Oct, most new logic (manual dispatch, failover, ratings) lands by internal evaluation (week 13), fire brigade and full feature set land by external evaluation (week 15), which is also the last date synthetic-only data is acceptable.
- Three items stay genuinely open (evaluation rubric format, real ambulance-org outreach ownership, whether org ratings ever become real) – everything else in the notes now has a concrete decision below.

Reconciliation: Blueprint vs. Supervisor Notes

Topic	Blueprint said	Supervisor add/change	notes Resolution
Ambulance request method	Part 2A only describes one path: "user requests ambulance via app" (automatic)	Notes add a second path: user calls the org directly by phone, and the org submits the request via the app on the user's behalf	Both paths ship in MVP. Manual submissions write a <code>ServiceRequest</code> row with <code>source = manual</code> through an org-facing form on the dispatcher dashboard, entering the same assignment pipeline as automatic requests (see Section 5)

Topic	Blueprint said	Supervisor add/change	notes Resolution
Cross-org co-operation	USP #2 already claims an “organization-agnostic” dispatch pool and “theoretical integration with Company X, Y, Z,” but never defines what happens if the assigned org is busy	Notes introduce an explicit MOU: participating orgs agree to hand off relief duty, and Madad+ auto-alerts the next-closest available org if the first is busy	Cross-org failover becomes a named, Must-have step in the dispatch algorithm (Section 5c), not an implied side effect of “organization-agnostic”
Ambulance selection criteria	Part 1 only ever says “closest available”; no mention of rating as a factor	Notes raise an open question: should non-urgent users get to pick a higher-rated ambulance over the closest one?	Closest-available stays the default always. A rating-filtered choice appears only for requests explicitly marked non-urgent, and only among options within a short distance/time ceiling of the closest one (full rule in Section 5d)
Admin/dispatcher scope	Part 2A scope: dashboard, map view, manual status override, request queue monitoring, basic admin panel	Notes add a second request-entry channel: the org itself submits on a caller’s behalf	Dispatcher dashboard gains a Must-have “submit request on behalf of caller” form in MVP, next to the existing status override (Section 3)
Emergency scope boundary	Part 2 discusses ambulances only for FYP; other emergency types (fire, police) only show up in Part C as post-FYP “expansion”	Notes explicitly confirm: MVP = ambulances only, fire brigade(s) added for the fuller FYP scope, before external evaluation	Fire brigade moves from “post-FYP future” into “full FYP, pre-external-eval” (Section 8). MVP boundary is unchanged from the blueprint’s original intent
“Quick Help” emergency subtypes	Part 1’s recommended-improvements list groups cardiac, accident, and fire as subtypes of one emergency request	Notes promote fire brigade to its own resource/organization type, separate from ambulances	Quick Help subtypes (cardiac, accident, general) apply only within the ambulance flow. Fire is its own request type with its own Organization/Resource records, not an ambulance subtype
Data authenticity plan	Part 1: “uses synthetic data for FYP demonstration... production would require MOUs and technical integration”	Notes confirm synthetic-only data specifically until external evaluation, with real company cooperation starting after that point	External evaluation (week 15) is the formal cutoff. No real-company integration work is scoped anywhere in this document before that date

Topic	Blueprint said	Supervisor add/change	notes	Resolution
Timeline milestones	/ No dates anywhere in the blueprint; only feature buckets (MVP/Optional/Future)	Notes add four concrete checkpoints: MVP Sept/1st week Oct, internal eval week 13, external eval week 15 (data cutoff)	four concrete MVP end ture bucket from Sections 3–4	Section 8 maps every feature bucket from Sections 3–4 onto these four checkpoints

1 Feature Breakdown – Emergency Services

1.1 MVP (Ambulances Only)

1.1.1 User-facing

Feature	Priority	Acceptance criterion
One-tap automatic ambulance request	Must	Tapping “Request Ambulance” from the Emergency tab creates a request with the user’s GPS location (or adjusted pin) in two taps or fewer
Urgent / non-urgent toggle	Must	User marks the request urgent (default) or non-urgent before submitting; this flag drives the selection rule in Section 5d
Live status tracking screen	Must	The tracking screen updates automatically through searching → assigned → en route → arrived → completed as the backend pushes each transition over Socket.IO
Assigned-ambulance details	Must	Once assigned, the user sees the organization name, vehicle ID, and a simulated ETA countdown
Manual-request tracking card	Should	If a request originated from the manual/org-submitted path and matches the user’s phone number on file, it appears in that user’s app with the same live tracking card as an automatic request
Request history entry	Should	Past ambulance requests, automatic or manual-originated, appear in Request History with timestamp and outcome
Quick Help subtypes (cardiac, accident, general)	Could	User can optionally tag the request with a predefined subtype before submitting, shown to the assigned org for context

1.1.2 Dispatch algorithm / backend logic

Feature	Priority	Acceptance criterion
Nearest-available selection (de-fault)	Must	For urgent or untagged requests, the system computes distance from every “available” ambulance across all participating orgs and assigns the minimum-distance one
Availability lock on assignment	Must	Assigning an ambulance immediately flips its status to busy and decrements that org’s available count, so no second request can claim it
Cross-org MOU failover	Must	If the assigned org marks itself busy or fails to confirm within 5 seconds, the system reassigns to the next-closest available ambulance from any other participating org, without a visible status flicker on the user’s screen
Manual-request ingestion end-point	Must	An org dispatcher can submit caller phone/address/urgency through a form, creating a <code>ServiceRequest</code> identical in shape to an automatic one, entering the same assignment pipeline
Status state machine enforcement	Must	The backend only accepts valid transitions (searching → assigned → en route → arrived → completed) and rejects any out-of-order jump
Simulated movement/ETA engine	Should	The assigned ambulance’s simulated coordinates move toward the user’s location at a fixed speed, and the displayed ETA counts down accordingly
Rating-filtered non-urgent selection	Should (target: internal eval)	For requests marked non-urgent, the system ranks available ambulances within a short ETA delta of the closest one by rating, and offers the top options instead of auto-assigning the single closest

1.1.3 Admin/dispatcher

Feature	Priority	Acceptance criterion
Map view of ambulances and requests	Must	Dispatcher sees every ambulance, color-coded by status, and every active request plotted on one map
Manual status override	Must	Dispatcher can toggle any ambulance between available/busy/offline, and the change reflects in the dispatch pool immediately

Feature	Priority	Acceptance criterion
Org-facing manual submission form	Must	Dispatcher can enter a caller’s phone number and address and submit a request on the caller’s behalf, without the caller needing the app
Request queue monitoring	Must	Dispatcher sees a live list of unassigned/searching requests sorted by wait time
Failover audit trail	Should (target: internal eval)	Dashboard shows which requests were reassigned across organizations due to failover, and which org ultimately handled each one

1.2 Full FYP Addition (Fire Brigade)

1.2.1 User-facing

Feature	Priority	Acceptance criterion
One-tap fire brigade request	Must	User selects “Fire” from the Emergency tab; the request flow mirrors the ambulance flow (location capture, tracking screen, status updates)
Fire requests are always urgent	Must (decision)	The urgent/non-urgent toggle and rating-based selection never appear for fire requests; fire dispatch always uses closest-available (rationale in Section 5d)

1.2.2 Dispatch algorithm / backend logic

Feature	Priority	Acceptance criterion
Nearest-available fire truck selection	Must	Same nearest-neighbor algorithm as ambulances, filtered to <code>resourceType = fireTruck</code>
Cross-org MOU failover for fire	Must	Same failover rule as ambulances (Section 5c), applied to fire-brigade organizations

1.2.3 Admin/dispatcher

Feature	Priority	Acceptance criterion
Fire resource layer on dispatcher map	Must	Dispatcher map shows fire trucks with a distinct marker/color from ambulances on the same map

Feature	Priority	Acceptance criterion
Fire org + fleet seed data	Must	At least 2 fictional fire-brigade organizations with a combined 6–10 trucks are seeded before the fire dispatch flow is demoed

2 Feature Breakdown – Domestic Services

No changes from the supervisor notes here. The MVP/full-FYP split below just formalizes the blueprint’s own Part A (must-have) vs. Part B (if time permits) distinction into this document’s priority structure.

2.1 MVP

Feature	Priority	Acceptance criterion
Service categories (Plumber, Electrician, Carpenter, AC Tech, Mechanic, Painter)	Must	User can browse and select from the 6 predefined categories on the Services tab
Synthetic provider database (5–10 per category)	Must	Each category has at least 5 seeded provider profiles with name, status, and a rating placeholder
Service request creation	Must	User can submit a request with category, free-text problem description, and a pinned or selected location
Provider assignment (round-robin)	Must	Backend assigns the first available provider in that category on a rotating basis, skipping any provider already marked busy
Simulated provider status updates	Must	An assigned domestic request shows the same status progression pattern as ambulance requests (assigned → en route → arrived → completed)
Active request tracking	Must	Domestic requests use the same active-request screen as emergencies, distinguished by request type
Request history	Must	Past domestic requests appear in the same history list as emergency requests, filterable by type

2.2 Full FYP (from blueprint Part B, if time permits)

Feature	Priority	Acceptance criterion
Ratings & reviews for providers	Should	User can rate a completed domestic job 1–5 stars; the average shows on the provider’s profile
Appointment scheduling	Should	User can pick a future date/time slot instead of “now” for a non-urgent domestic job
Pricing display	Should	The category or provider screen shows an indicative price range before booking
Multi-language (Urdu/English)	toggle Could	Settings screen lets the user switch the app’s language
In-app chat with provider	Could	User and assigned provider can exchange text messages tied to the active request
Separate provider mobile app	Could	Providers get their own lightweight app instead of managing status through the admin panel

3 Ambulance Dispatch Logic – Detailed Spec

3.1 Automatic path (in-app request)

1. User opens the Emergency tab and taps “Request Ambulance.”
2. App captures GPS location (or the user adjusts a pin) and asks for the urgent/non-urgent flag (urgent is the default).
3. App creates a `ServiceRequest` (`requestType = emergency`, `emergencyResourceType = ambulance`, `source = automatic`, `status = searching`).
4. The dispatch algorithm runs across every participating org’s available ambulances (rule in Section 5d).
5. On a match, the system creates an `Assignment` linking request to ambulance, flips that ambulance to busy, decrements the org’s available count, and sets `status = assigned`.
6. The winning org’s dispatcher dashboard shows the new assignment. For the FYP simulation, this either auto-confirms after a short delay or a dispatcher can override within the grace window.
7. If the org fails to confirm, or marks itself unavailable within 5 seconds, cross-org failover runs (below).
8. Once confirmed, the simulated movement/ETA engine starts and `status → en route`.
9. When the simulated ambulance’s coordinates reach the user’s location, `status → arrived`.
10. The dispatcher (or a fixed on-scene timer, for demo purposes) marks the job done: `status → completed`, the ambulance flips back to available, and the org’s count increments.

3.2 Manual path (phone call → org submits via app)

1. User calls the ambulance organization’s number directly, the same way they would today, bypassing the app entirely.
2. The org’s phone operator takes the caller’s details exactly as they normally would.
3. The operator or dispatcher logs into the org-facing panel of the Madad+ dispatcher dashboard and submits a manual-request form: phone number, address/pin, urgency.
4. This creates a `ServiceRequest` with `source = manual` and `submittedByOrgId` set to that org. Because the caller already chose this org by phoning them, the request skips the cross-org search and goes straight to that org’s own fleet.
5. That org’s available-ambulance count decrements immediately, exactly like the automatic path, so the same vehicle can’t be double-booked by an unrelated automatic request.
6. If the caller’s phone number matches a Madad+ account, the request automatically appears in that user’s app under Active/History. If not, the request exists only on the org/admin side, for fleet accounting.
7. From here, the request follows the identical state machine as automatic (assigned → en route → arrived → completed), and cross-org failover still applies if that org’s own ambulance becomes unavailable before confirming. The MOU means even a manually-called org can hand off to the next-closest org rather than leaving the caller waiting.

3.3 Cross-org MOU failover (busy-on-assignment)

1. Trigger: the assigned org marks itself busy/unavailable, or doesn’t confirm within a 5-second grace window (chosen for FYP demo pacing, adjustable later), before status reaches en route.
2. The failed assignment is removed. The ambulance reverts to whatever its honest status actually is (it does not get double-decremented).
3. The dispatch algorithm re-runs, excluding the org that just failed, across every remaining participating org’s available resources, applying the same urgency-based selection rule as the original request.
4. The next-closest available ambulance is assigned. The user-facing screen does not flicker back to “searching” – the reassignment happens behind the scenes and the user only sees “assigned” once it resolves.
5. The failover event is logged (which org failed, which org picked up) for the dispatcher dashboard’s audit trail.
6. **[Recommended Addition]** If no org has any available resource, the request stays in searching with a “still finding an ambulance” state and a retry loop, escalating to the dispatcher dashboard after a fixed number of attempts. The notes don’t specify this case, but a dispatch system without it breaks visibly in a demo the moment every ambulance is busy at once.

3.4 Closest vs. rating – resolution

Decision: Closest-available is the default for every ambulance request, full stop. A rating-influenced choice only appears when the user explicitly marks the request non-urgent, and even then it’s limited to ambulances within a short ETA delta of the closest option.

- **Urgent (default):** always assign strictly by minimum distance among available ambulances. No rating shown, no choice offered. Every extra second of deliberation is a liability risk, not a feature.
- **Non-urgent (explicit toggle):** compute the set of available ambulances within roughly 5 extra minutes of ETA compared to the closest one, rank that subset by rating, and present the top 2–3 options (for example, “Closest – Al-Khidmat, 6 min” vs. “Highest rated – Chhipa, 9 min”). If the user doesn’t pick within a short timeout, default to closest.

Rationale: rating-based choice is only safe to offer once the user has told the system, by their own hand, that time pressure is lower. Even then, pre-filtering by a distance/time ceiling means “highest rated” can never mean “20 minutes away” for something the user already called non-urgent for a reason. The urgent path stays foolproof by having no decision to make.

3.5 Status state machine

From	To	Trigger
(none)	Searching	User submits an automatic request, or an org submits a manual one without a pre-assigned ambulance
Searching	Assigned	Dispatch algorithm finds an available ambulance (closest, or rating-filtered per the rule above) and creates the Assignment
Assigned	Searching (internal reassignment, not shown to the user)	Cross-org failover trigger: assigned org goes busy or fails to confirm within the grace window
Assigned	En route	Org confirms (automatic path), or the operator confirms directly since they already committed by phone (manual path); the simulated movement engine starts
En route	Arrived	Simulated ambulance coordinates cross a small distance threshold from the user’s location
Arrived	Completed	Dispatcher marks the job done, or a fixed on-scene timer elapses for demo purposes

4 Tech Stack – Validation & Decisions

None of the six layers change. The new notes don’t introduce anything a Flutter/Node/Postgres stack can’t handle on a 3-person team’s 4-week MVP clock. What they do force is two schema

decisions that need to happen now, before MVP code is written, so the fire-brigade addition and the rating logic don't require a rebuild later.

Layer	Decision	Reasoning
Flutter (mobile)	Confirmed	One codebase still matches a 3-person team on a tight timeline. New requirement: build the Emergency tab against a generic resourceType (ambulance/fireTruck) from the start, not a hardcoded "ambulance screen," so fire brigade becomes a data/UI variant instead of a rewrite
Node.js + Type-Script	Confirmed	Manual dispatch is just another authenticated route into the same Express/Fastify service layer; no second backend service needed for a 3-person team to maintain
PostgreSQL + PostGIS	Confirmed, two schema decisions locked in now	ServiceRequest needs source (automatic/manual) and urgency (urgent/nonUrgent) columns from the first migration. Organization/Resource need a generic orgType/resourceType column and an Organization.rating field, because both the rating-vs-closest logic and the fire-brigade addition depend on this shape existing before those features get built. The synthetic-data-only constraint means these fields can be hand-seeded rather than computed from real usage
Google Maps Platform (free tier)	Confirmed	Manual requests already carry an address from the phone call; fire brigades just add another marker layer on the same dispatcher map. No new mapping capability required
Socket.IO	Confirmed	Failover reassignment and manual-request status updates broadcast through the same request-scoped rooms already planned for automatic requests; no second real-time channel needed
Firebase Auth + JWT	Confirmed for customers, one addition	Org dispatcher/operator accounts (needed to submit manual requests) use the custom-JWT-plus-bcrypt path the blueprint already listed as Firebase's alternative, kept separate from customer Firebase Auth. Staff accounts are internal and few in number, so adding Firebase accounts for them is unnecessary overhead for a 3-person team

5 Data Model Sketch

Entities are shaped so that adding fire brigades doesn't require touching the **Organization** or **ServiceRequest** tables again. **resourceType** and **orgType** carry that weight now.

Entity	Key fields	Relationships
User	userId, phone, name, role	One user has many ServiceRequests as requester

Entity	Key fields	Relationships
Organization	orgId, name, orgType (ambulanceProvider / fireBrigade), rating, contactPhone, isActive	One org has many Resources; one org can submit many ServiceRequests (manual path)
Resource	resourceId, orgId (FK), resourceType (ambulance / fireTruck), status (available/busy/offline), currentLat, currentLng	Belongs to one Organization; has at most one active Assignment
Provider	providerId, category (plumber / electrician / carpenter / acTech / mechanic / painter), name, rating, status	Has many Assignments over time (domestic jobs)
ServiceRequest	requestId, userId (FK, nullable), requestType (emergency/domestic), emergencyResourceType (nullable), domesticCategory (nullable), source (automatic/manual), urgency (nullable), status, locationLat, locationLng, address, submittedByOrgId (FK, nullable), createdAt	Optionally belongs to one User; optionally submitted by one Organization; has 0 or 1 Assignment
Assignment	assignmentId, requestId (FK), resourceId (FK, nullable), providerId (FK, nullable), assignedAt, confirmedAt, failoverFromOrgId (nullable), completedAt	Belongs to one ServiceRequest; references either one Resource or one Provider, never both
Rating	ratingId, requestId (FK), ratedEntityType (organization/provider), ratedEntityId, stars, comment, createdAt	Belongs to one ServiceRequest; references either one Organization or one Provider

Domestic providers stay a separate, lighter entity from **Organization/Resource** on purpose – they’re individual people, not fleets, and forcing them into the same generic model would be an abstraction with no real use yet.

6 Phased Roadmap

Dates below anchor “13 weeks from now” and “15 weeks from now” to today (Sep 6, 2026): internal evaluation lands around the first week of December, external evaluation about two weeks after that. Adjust if the notes meant a different reference date.

Feature/milestone	Phase	Why this phase
Automatic ambulance request (urgency flag, GPS/pin, closest-available default)	MVP	Core USP; nothing else in the app works without it

Feature/milestone	Phase	Why this phase
Cross-org MOU failover on busy	MVP	Small addition on top of the same nearest-neighbor query; central to the “not just closest overall” pitch
Manual dispatch ingestion (org-facing submission form)	MVP	Supervisor named this as one of two required ways to trigger dispatch; both must exist before any demo
Ambulance status state machine	MVP	Every other emergency feature reads or writes this
Domestic services: categories, request creation, round-robin assignment	MVP	Second half of the two-tab USP; the “one app for all help” pitch means nothing without it
Dispatcher dashboard: map, manual override, request queue	MVP	Needed to demo dispatch working and to run the manual-submission form
Synthetic seed data (3–5 ambulance orgs / 10–15 ambulances, 5–10 providers per domestic category)	MVP	Nothing above functions without seed data; sizing already set by the blueprint
User auth, profile, basic request history	MVP	Baseline account plumbing every other feature depends on
Rating-based non-urgent ambulance selection Organization.rating seed-ing	Internal evaluation + tion	Needs the rating field and a comparison UI on top of the MVP dispatch engine; not required for the first working demo
Failover/manual-dispatch audit trail on dispatcher dashboard	Internal evaluation	Useful to show, but not required to prove the core loop works at MVP
Domestic provider ratings & reviews	Internal evaluation	Blueprint listed this as optional-if-time-permits; reasonable next slice after MVP
In-app notifications	Internal evaluation	Blueprint core feature, but the app still functions for an MVP demo without it
Basic admin panel (users, providers, organizations)	Internal evaluation	Useful for showing scale of seed data; not required to prove the dispatch algorithm
Fire brigade dispatch (mirrors ambulance flow on the generic Resource model)	External evaluation	Supervisor scoped this to “full FYP,” which lands at the external-eval checkpoint, not MVP
Dispatcher map: fire resource layer + fire org seed data	External evaluation	Follows directly from adding fire as a resource type

Feature/milestone	Phase	Why this phase
Appointment scheduling for domestic services	External evaluation	Blueprint Part B item; adds value but isn't core to either USP
Multi-language (Urdu/English) toggle	External evaluation	Blueprint Part B item; accessibility polish, not core logic
Pricing display for domestic categories	External evaluation	Rounds out the domestic-services story for a full external-facing demo
Real ambulance/fire-org outreach and integration	Beyond external eval	Notes explicitly place real cooperation after the synthetic-data cutoff, not before
Remaining Could-priority items (chat, provider app, push notifications)	Beyond external eval stretch	Blueprint always flagged these as optional-if-time-permits; lowest claim on a 3-person team's remaining hours

7 Risks & Mitigations

Risk	Status	Mitigation
Liability & expectations – users think this is a real emergency service	Updated	Now also covers the MOU: the “cooperating organizations” are fictional for the FYP, not real signed partners. Extend the existing “SIMULATION MODE” disclaimer to explicitly cover the MOU and failover behavior
Ambulance-org integration reality	Updated	Notes confirm real integration is a post-external-eval activity. Nothing changes before week 15; state this cutoff plainly in the FYP report so evaluators don't expect real API access at MVP or internal eval
Service quality control (domestic)	Unchanged	Same mitigation from the blueprint (verification, ratings, warranty policy), now formally scheduled at internal/external eval per the roadmap above
Address accuracy in Karachi	Unchanged, scope widened	Same landmark/manual-pin mitigation, now also applied to the manual-dispatch form – an operator typing an address by hand carries the same risk as a user dropping a pin
Internet reliability	Unchanged	Same offline-caching/minimal-data approach; SMS fallback stays a future item

Risk	Status	Mitigation
MOU/failover realism	New	The cross-org cooperation and busy-handoff behavior is entirely simulated between fictional organizations. State this explicitly wherever the MOU is described in reports or demos
Manual-dispatch data integrity	New	A manual request depends on an org’s own staff entering it honestly. An org could under-report its busy ambulances to look more available, or the reverse. Mitigate by logging every manual submission with a timestamp and submitting-org ID on the audit trail (Section 3)
Rating-gaming (future)	New	Organization ratings are synthetic for the FYP, so there’s no real fraud risk yet. Once real orgs onboard post-external-eval, self-inflated ratings become a real risk – flag this now as a production-phase requirement, not something the FYP needs to solve
Timeline compression	New	MVP is due in about four weeks, and auth, the dispatch algorithm, real-time tracking, the dispatcher dashboard, and domestic services all land in that same window for a three-person team. The Must-only MVP list in Section 8 deliberately defers rating logic, audit trails, and fire brigades so the four-week window covers only what’s strictly necessary to prove the core loop

Remaining Open Items

- **Evaluation deliverable format and grading rubric** for internal (week 13) and external (week 15) evaluations – this depends on the department’s specific FYP process, which isn’t covered by the blueprint or the notes provided here.
- **Ownership and timeline for real ambulance/fire-org outreach** after the synthetic-data cutoff – whether Edhi, Chhipa, 1122/SIEHS, JDC, or KMC Fire are willing to cooperate is outside the team’s or supervisor’s unilateral control.
- **Whether organization ratings ever become real, user-generated scores post-integration**, or stay a platform-curated trust signal even in production – a business-model fork the team hasn’t made yet, and one that only becomes decidable once real-org talks actually start.